Algorithmic Prompt Search for a Domain-Specific Academic Advising LLM

AI Project Report: Washington State University

Team Members: Jaime Gudino, Andrew Edson, Joseph Buchanan

Spring 2026

Introduction and Problem Statement

Large Language Models (LLMs) have demonstrated impressive capabilities in natural language understanding and generation, yet their performance on domain-specific, factually grounded tasks is highly sensitive to prompt formulation. Current prompt engineering is manual, slow, and rarely optimal. At the same time, WSU students using the Virtual Counselor web application we are independently developing currently have no way to ask natural-language questions about their degree progress: questions such as *"What courses do I need before CPTS 360?"* or *"Which UCORE requirements have I not yet satisfied?"*

This project addresses both challenges simultaneously:

1. **Research question:** Can algorithmic search over a structured prompt mutation space automatically discover effective prompts, reducing reliance on hand-engineering?
2. **Practical application:** Build and evaluate a RAG pipeline that allows WSU students to query the course catalog, prerequisite chains, schedule availability, and degree requirements using plain English.

The two goals are tightly coupled: the same prompt-scoring infrastructure that drives the search algorithms also evaluates RAG context strategies, providing a unified empirical framework for the whole system.

Solution Overview

Our system consists of three layers that build on each other (Figure 1):

Layer 1: Retrieval A FAISS vector index over the WSU course catalog is combined with a live SQLite database containing sections, seat counts, prerequisites, and degree requirements. At query time, a hybrid retriever fetches the k most semantically relevant course chunks, and a cross-encoder reranker (NvidiaReranker) re-scores them before injection.

Layer 2: Context Assembly A `ContextBuilder` assembles a multi-block prompt that contains (a) a UCORE definition block, (b) a degree requirement summary, (c) a 2-hop BFS prerequisite chain, (d) live schedule data, and (e) few-shot Q&A examples, each block included only when the query semantically requires it.

Layer 3: Prompt Search Three complementary algorithms search the space of *prompt templates* by applying mutation operators (add chain-of-thought trigger, add verification step, add

self-consistency instruction, etc.) and scoring each candidate on a held-out validation set.

React Frontend (ChatPage)	Node.js / Express (LLM endpoints)	RAG Pipeline (Python src/)	SQLite + FAISS (course catalog)

Figure 1: High-level system architecture. The React frontend sends student queries to an Express backend, which delegates to the Python RAG pipeline backed by FAISS and SQLite.

Technical Implementation

Retrieval-Augmented Generation Pipeline

Data sources. The course catalog is ingested from WSU's public schedule pages via automated n8n workflows and stored in a SQLite database (courses, sections, degree requirements) alongside a FAISS flat-index over 384-dimensional sentence-transformer embeddings (`all-MiniLM-L6-v2`). A separate degree ingestor parses the free-text CS degree requirement block from the catalog.

Retrieval. The `CourseRetriever` converts the student query to an embedding, fetches the top-30 candidate chunks from FAISS (configurable via `reranker_fetch_k`), and then the `NvidiaReranker` cross-encoder re-ranks them to the final top-3 using semantic relevance rather than raw cosine distance.

Context assembly. The `ContextBuilder` composes the prompt from up to five specialized blocks:

- **UCORE block**: Injected whenever the query mentions UCORE categories (WRTG, QUAN, DIVR, etc.). Contains all 10 category definitions so the LLM can reason about unfulfilled requirements.
- **Degree block**: Pulled from the SQLite degree table when the query involves credit counts, graduation tracking, or a degree audit.
- **Prerequisite block**: A 2-hop BFS over the catalog prerequisite graph starting from course codes mentioned in the query. Surfaces the full upstream dependency chain (e.g., CPTS 360 ← CPTS 223, CPTS 260 ← CPTS 122, MATH 216).
- **Schedule block**: Live section times and seat availability queried from the SQLite sections table, included when the question asks about enrollment, conflicts, or meeting times.
- **Few-shot block**: Three exemplar Q&A pairs loaded from `few_shot_examples.json` to anchor the LLM response style.

The LLM used for generation is **Claude Haiku** (`claude-haiku-4-5`), accessed via the Anthropic API. A local fallback using **Llama 3.1-8B** (Q4_K_M GGUF quantization via `llama-cpp-python`) was also evaluated on an Apple M4 Mac mini, achieving 35–45 tokens/second on CPU with comparable answer quality on the WSU domain subset.

Prompt Template System

Prompts are represented as structured `PromptTemplate` objects with six independently mutable fields:

- `system_role`: persona string (e.g., "WSU academic advisor")
- `task_description`: free-form instruction paragraph
- `cot_trigger`: chain-of-thought preamble
- `examples`: list of few-shot Q&A pairs

- `output_format`: format specification (prose, JSON, numbered list)
- `constraints`: domain rules (e.g., "Use WSU course codes")

A `PromptMutator` provides ten atomic operators that clone and modify a template (each appending to a `history` list so the derivation path is traceable); four are active in all search runs (`add_cot`, `remove_cot`, `add_verification_step`, `add_self_consistency`), with the remaining six available for selective use:

- CoT control: `add_cot`, `remove_cot`
- Answer quality: `add_verification_step`, `add_self_consistency`
- Structure: `add_expert_persona`, `add_example`, `set_output_format`
- Domain: `add_domain_context`, `add_constraint`, `rephrase_task`

Search Algorithms

3.3.1 Beam Search

`BeamSearchPromptOptimizer` maintains a beam of the top-B prompt candidates (default $B = 3$). At each iteration, every beam member is expanded by applying all mutation operators; duplicates are deduplicated by their mutation path string. Each candidate is scored by running it against the validation set, and the top-B scoring candidates advance. Early stopping applies after *patience*=2 consecutive iterations without improvement exceeding *min_delta*. The search terminates after at most $I_{\max} = 10$ iterations.

Formally, let $\mathcal{B}_t$ be the beam at iteration t and $f(\pi, \mathcal{D})$ the accuracy of template π on validation dataset $\mathcal{D}$:

$$\mathcal{C}_t = \{m(\pi) \mid \pi \in \mathcal{B}_{t-1},\ m \in \mathcal{M}\}, \quad \mathcal{B}_t = \operatorname{top}_B\big(f(\cdot, \mathcal{D})\big)(\mathcal{C}_t)$$

3.3.2 Monte Carlo Tree Search (MCTS)

`MCTSPromptSearch` builds a search tree where each `MCTSNode` stores a prompt template, visit count, and cumulative reward. Each of the $N = 100$ iterations executes four phases:

1. **Selection**: traverse from root using UCB1: $\mathrm{UCB1}(v) = \frac{w_v}{n_v} + c\sqrt{\frac{\ln n_{\text{parent}}}{n_v}}$, with exploration weight $c = \sqrt{2} \approx 1.414$.
2. **Expansion**: attach one new child via a randomly chosen mutation.
3. **Simulation**: evaluate the child on the validation set; reward = accuracy.
4. **Backpropagation**: update visits and total reward for all ancestors.

At the end, the root child with the highest average reward is selected as the best template. MCTS trades per-iteration depth (only one mutation per expansion) for broader exploration of the mutation tree via UCB1-guided selection.

3.3.3 Iterative Self-Refinement

`IterativeRefinementOptimizer` takes a generate–critique–revise loop: the LLM generates an answer with the current prompt, a separate critique call identifies what went wrong, and the prompt is modified based on the critique. This runs for up to $R = 5$ rounds. Unlike beam search and MCTS, refinement uses the LLM's own feedback signal rather than a held-out accuracy score, making it potentially more data-efficient but also more sensitive to the quality of the critique.

Evaluation Framework

The primary metric is **semantic similarity accuracy**: an answer is marked correct if the cosine similarity between its `all-MiniLM-L6-v2` embedding and the ground-truth embedding exceeds 0.60. This tolerates paraphrasing and partial answers without requiring exact string matching. The metric falls back to substring containment if `sentence-transformers` is unavailable.

Secondary metrics include **Pass@k**, **token cost** (Haiku pricing: $0.00025/1K input, $0.00125/1K output), and **normalized improvement over baseline**.

Datasets

WSU Domain Test Suite (120 Cases)

We constructed 120 manually authored, catalog-verified test cases spanning six categories that mirror realistic advising queries:

Category	Count	Example query
Prerequisite validation	30	Can I take CPTS 122 if I have CPTS 121?
Prerequisite chain discovery	25	What courses must I complete before CPTS 360?
Credit calculations	20	How many credits remain if I have completed 30?
Degree progress analysis	20	What's left for my CS degree given [courses]?
Schedule feasibility	15	Can CPTS 360 and CPTS 440 fit in one semester?
UCORE planning	10	Which UCORE categories have I not satisfied?

Table 1: WSU domain test suite breakdown.

All expected answers were validated against WSU's official catalog database and degree requirement documents. Test case authoring revealed several catalog inconsistencies (notably 15 of 25 chain discovery cases initially had incorrect expected chains); these were corrected after cross-referencing the live catalog.

Standard Reasoning Benchmarks

To anchor our prompt search evaluation in published results we also curated:

- **GSM8K**: 500 grade-school math problems from the full 8,500-case set.
- **MATH**: 200 competition math problems across five difficulty levels.

These allow direct comparison of our beam search and MCTS prompt optimizers against established baselines on well-understood reasoning tasks where ground truth is unambiguous.

Empirical Results

RAG vs. No-RAG Ablation

The most significant empirical finding is the impact of the RAG pipeline on domain-specific accuracy. Table 2 shows the full breakdown across all six categories using Claude Haiku as the generation model.

Category	n	RAG Accuracy	No-RAG Accuracy	Δ
Prerequisite validation	30	**93.33%**	16.67%	+76.67 pp
Prerequisite chain discovery	25	**88.00%**	28.00%	+60.00 pp
Schedule feasibility	15	**86.67%**	26.67%	+60.00 pp
UCORE planning	10	**80.00%**	50.00%	+30.00 pp
Credit calculations	20	**80.00%**	55.00%	+25.00 pp
Degree progress analysis	20	**55.00%**	25.00%	+30.00 pp
Overall	120	**81.67%**	30.83%	**+50.83 pp**

Table 2: RAG vs. no-RAG semantic similarity accuracy on the 120-case WSU domain test suite. Run date: 2026-04-27. LLM: claude-haiku-4-5. Threshold: cosine ≥ 0.60.

Overall, the RAG pipeline achieved 81.67% accuracy (98/120 cases) compared to 30.83% without retrieval, a gain of 50.83 percentage points.

Per-Category Analysis

Prerequisite validation (93.3%) is the strongest category because the prerequisite block injects exact catalog text (e.g., "CPT S 122 requires CPTS 121 with a C or better") directly into the prompt, leaving the LLM with little ambiguity.

Chain discovery (88.0%) improved dramatically from an earlier 40% figure. Two fixes drove this: (a) correcting 15/25 test cases against the live catalog, and (b) upgrading the prerequisite retriever from 1-hop to a 2-hop BFS that surfaces the full upstream chain (e.g., CPTS 360 requires CPTS 223 and CPTS 260; CPTS 223 requires CPTS 122; CPTS 122 requires CPTS 121).

Schedule feasibility (86.7%) is the most improved category relative to earlier runs. Tighter output format constraints in the few-shot block reduced the verbosity that previously caused embedding mismatch against short expected answers.

UCORE planning (80.0%) benefits from the static UCORE category block, which provides all ten category definitions the LLM needs to enumerate unfulfilled requirements. Without this block, the LLM hallucinates category names.

Credit calculations (80.0%) is well served by the degree block, which injects the student's credit totals directly, allowing the LLM to perform simple arithmetic rather than relying on parametric knowledge.

Degree progress (55.0%) is limited by the free-text nature of the degree requirement block: the catalog encodes requirements as prose that the ingestor parses imperfectly. Several artifact strings ("PASS CS 121", "TAKE MATH 171") appear in the parsed requirement list, reducing the LLM's ability to match courses correctly.

Local Model Evaluation

To assess deployment cost, we tested **Llama 3.1-8B-Instruct** (Q4_K_M quantization, 4.65 GB) on an Apple M4 Mac mini via `llama-cpp-python`. The model ran at **35–45 tokens/second** on CPU, producing answers of comparable factual quality to Claude Haiku on the 5-question sample suite (prerequisite, chain, credit, UCORE, degree categories). The 3B fallback model (Llama

3.2-3B-Instruct, Q4_K_M) was also tested on a GTX 950 (2 GB VRAM) for comparison. Local inference eliminates API costs entirely but requires 8+ GB RAM and accepts higher latency (4–6 seconds per response vs. ∼1 second for Haiku).

Prompt Search Algorithms

The beam search, MCTS, and iterative refinement implementations are fully functional and tested against a mock LLM stub. Due to API budget constraints during the project window, live evaluation of the search algorithms against the full GSM8K and MATH benchmarks was not completed; that run is staged and ready. The WSU domain evaluation (notebook 15) represents the primary live-API result.

Table 3 summarizes the design parameters of the three algorithms:

Algorithm	Beam / Iterations	API calls (est.)	Exploration strategy
Beam Search	$B = 3$, $I = 10$	$B \times \lvert\mathcal{M}\rvert \times I \times \lvert V\rvert$	Greedy top-B pruning
MCTS	100 iterations	$N \times \lvert V\rvert$	UCB1 (exploit + explore)
Iterative Refinement	5 rounds	$R \times (\lvert V\rvert + 1)$	LLM self-critique

Table 3: Prompt search algorithm parameters. $\mathcal{M} = 4$ active mutations, N = MCTS iterations, V = validation set, R = refinement rounds. Each candidate evaluation costs $|V|$ API calls; refinement adds one critique call per round.

Key Insights

1. **Structured retrieval beats semantic retrieval alone for factual lookups.** The prerequisite BFS block (a structured graph traversal over the catalog DB) was more impactful than FAISS similarity search for prerequisite questions. Semantic retrieval shines for open-ended description questions; structured DB queries win for binary yes/no prerequisite decisions.

2. **Context specificity matters more than context volume.** The conditional block injection strategy (only include the UCORE block for UCORE questions, only include the degree block for progress questions) outperformed earlier versions that included all blocks unconditionally. Irrelevant context increases prompt length and confuses the LLM.

3. **Output format constraints directly close the verbosity gap.** Early runs showed schedule feasibility as the weakest category (53.3%) because the LLM produced multi-paragraph markdown whose embedding did not score ≥ 0.60 against short expected answers. Adding concise few-shot format anchors raised this category to 86.7% in the final run, confirming that format mutations (e.g., `set_output_format`) are high-leverage targets for the prompt search algorithms.

4. **Algorithmic prompt search and RAG are complementary, not competing.** RAG improves accuracy by grounding answers in retrieved facts; prompt search improves accuracy by finding the template structure that best helps the LLM use those facts. The two-level optimization (what to retrieve and how to prompt) is more powerful than either alone.

5. **Local models are a viable zero-cost alternative.** Llama 3.1-8B at 35–45 tok/s on M4 hardware produces answers on par with Claude Haiku for straightforward prerequisite and

credit queries. For a student-facing application with a self-hosted deployment budget, local inference is a realistic production path.

Related Concepts and Course Connections

This project applies several topics from the course:

- **Heuristic Search (Beam Search)**: The prompt optimizer directly applies beam search: a bounded-width best-first search with a learned heuristic (validation accuracy).
- **Monte Carlo Methods (MCTS)**: UCB1-guided tree exploration with simulation-based reward estimation mirrors MCTS as taught in the context of game playing and planning.
- **Representation & Embeddings**: Sentence-transformer embeddings serve both as the retrieval index and as the evaluation signal (cosine similarity metric).
- **Expert Systems / Knowledge Bases**: The structured prerequisite graph and catalog DB function as a domain knowledge base, with the LLM acting as the inference engine.
- **Prompt Engineering as State Space Search**: Framing prompt templates as discrete states connected by mutation operators reduces prompt optimization to a classical AI search problem.

Conclusion

We built a complete RAG-augmented LLM pipeline for WSU academic advising and evaluated it on a 120-case domain-specific test suite. The RAG pipeline achieves **81.67% semantic similarity accuracy** compared to 30.83% for an LLM without retrieval, a 50.83-percentage-point improvement driven primarily by structured prerequisite graph traversal and conditional context injection.

On the research side, we implemented three algorithmic prompt search strategies (beam search, MCTS, and iterative refinement) on a structured prompt template representation with 10 mutation operators. These algorithms provide a principled, reproducible alternative to manual prompt engineering and are positioned to further improve accuracy once run against live API endpoints on the GSM8K and MATH benchmarks.

The system demonstrates that combining structured knowledge retrieval with algorithmic prompt search produces a practically useful and academically grounded AI application: one that directly benefits WSU students while advancing understanding of how to systematically optimize LLM behavior for complex reasoning tasks.

Appendix: Code Structure

```
prompt-search/
  src/
    llm/            # Claude Haiku + local Llama clients
    retrieval/      # FAISS retriever, ContextBuilder, DB client, reranker
    prompts/        # PromptTemplate, PromptMutator
    search/         # BeamSearch, MCTS, IterativeRefinement
    evaluation/     # EvaluationMetrics (semantic similarity, pass@k, cost)
    counselor/      # Pipeline, PrereqChecker, GradAdvisor
  notebooks/
    core/           # 00-07: LLM setup, templates, search, RAG
```

```
  evaluation/     # 10-15: Benchmark loading and evaluation runs
  experiments/    # 20-21: Ablation, self-consistency
data/
  domain/         # 120 WSU test cases, FAISS index, few-shot examples
  results/        # rag_eval_results.json (authoritative)
  benchmarks/     # gsm8k_subset.json, math_subset.json
scripts/          # build_index.py, run_rag_eval.py, test_local_model.py
```